BeaconKit 아키텍처 분석 - 01. 개요 및 전체 아키텍처

1. BeaconKit 개요

BeaconKit은 **EVM 기반 합의 클라이언트를 구축하기 위한 모듈형 프레임워크**입니다. 이더리움 메인넷과 기능적으로 동일한 실행 환경을 보장하면서도 가장 사용자 친화적인 방식으로 EVM 블록체인을 구축하고 운영할 수 있도록 설계되었습니다.

1.1 핵심 특징

- **모듈형 설계**: 각 컴포넌트가 독립적으로 구성되어 유연한 확장 가능
- **CometBFT 기반 합의**: 이더리움의 LMD-GHOST + Casper FFG 대신 Tendermint 합의 알고리즘 사용
- **Engine API 지원**: 모든 주요 이더리움 실행 클라이언트와 호환
- **실시간 블록 생성**: 고정된 12초 슬롯 대신 타임아웃 기반 라운드 시스템
- **Byzantine Fault Tolerance**: 최대 1/3 악의적 검증자까지 허용

1.2 지원 실행 클라이언트

Engine API를 통해 다음 실행 클라이언트들을 지원합니다:

- **Bera-Geth**: Berachain 프로토콜의 공식 Go 구현체
- **Bera-Reth**: 성능과 안정성에 중점을 둔 Rust 기반 클라이언트

2. 전체 시스템 아키텍처

2.1 시스템 구성도

```
┌──────────────────────────────────────────────────────────────────────┐
| Service Registry |
| (모든 서비스의 생명주기 관리자) |
| 시작 순서: |
| 1. ShutdownService → 2. ValidatorService → 3. NodeAPIServer |
| 4. ReportingService → 5. TelemetryService → 6. EngineClient |
| 7. ChainService → 8. CometBFTService |
└──────────────────────────────┬───────────────────────────────────────┘
| 생명주기 관리
▼
┌──────────────────────────────────────┐
| CometBFT Service |
| (P2P를 통한 합의 조정자) |
| ┌──────────────────────────────┐ |
| | ABCI++ 인터페이스: | |
| | • Info | |
| | • InitChain | |
| | • PrepareProposal | |
| | • ProcessProposal | |
| | • FinalizeBlock | |
| | • Commit | |
| └──────────────────────────────┘ |
```

```
└──────────────┬──────────────────────┘
               │
PrepareProposal │ ProcessProposal/FinalizeBlock
┌──────────────┴──────────────┐
▼                             ▼
┌──────────────────────┐  ┌──────────────────────┐
│ Validator Service    │  │ Blockchain Service   │
│ (블록 빌더)          │  │ (블록 처리기)        │
│ ┌──────────────────┐ │  │ ┌──────────────────┐ │
│ │ • StateProcessor│ │  │ │• StateProcessor│ │
│ │ • BlobFactory   │ │  │ │• BlobProcessor │ │
│ │ • PayloadBuilder│ │  │ │• LocalBuilder  │ │
│ │ • Signer        │ │  │ │• DepositFetcher│ │
│ └──────────────────┘ │  │ └──────────────────┘ │
└──────────┬───────────┘  └──────────┬───────────┘
           │                         │
           │ 둘 다 사용              │ 둘 다 사용
           ▼                         ▼
┌──────────────────────────────────────────┐
│ Execution Engine                         │
│ (Engine API 클라이언트 래퍼)             │
│ • forkchoiceUpdate                       │
│ • newPayload / getPayload                │
```

```
└──────────────┬──────────────────────────────┘
               │ 통신
               ▼
외부 EL 클라이언트
(Geth/Reth/등)
```

2.2 핵심 데이터 흐름

블록 생성 흐름

1. **CometBFT** → PrepareProposal → **Validator Service**
2. **Validator Service** → 페이로드 빌딩 → **Execution Engine**
3. **Execution Engine** → 외부 EL 클라이언트와 통신
4. **Validator Service** → 블록 조립 → **CometBFT**

블록 처리 흐름

1. **CometBFT** → ProcessProposal/FinalizeBlock → **Blockchain Service**
2. **Blockchain Service** → 상태 전환 → **State Processor**
3. **State Processor** → 스토리지 업데이트 → **Storage Backend**
4. **Blockchain Service** → 실행 엔진 업데이트 → **Execution Engine**

3. 주요 디렉토리 구조

3.1 핵심 컴포넌트 디렉토리

```
beacon/ # 비콘 체인 핵심 로직
```

```
├── blockchain/ # 블록 처리 및 상태 전환 서비스
├── validator/ # 블록 빌딩 및 검증 서비스
└── payload-time/ # 페이로드 타이밍 관리

chain/ # 체인 사양 및 헬퍼
├── chain_ids.go # 체인 ID 정의
├── spec.go # 체인 사양 인터페이스
└── helpers.go # 체인 관련 유틸리티

consensus/ # 합의 레이어
├── cometbft/ # CometBFT 통합
└── types/ # 합의 타입 정의

execution/ # 실행 클라이언트 통합
├── client/ # Engine API 클라이언트
├── engine/ # 실행 엔진 구현
└── deposit/ # 예치 계약 모니터링

node-core/ # 노드 인프라
├── components/ # DI 컴포넌트 제공자
├── services/ # 서비스 레지스트리
└── builder/ # 노드 빌더
```

```
storage/ # 다층 저장소
├── beacondb/ # 비콘 상태 저장소
├── blockstore/ # 블록 저장소
├── availabilitystore/ # 블롭 저장소
└── depositstore/ # 예치 이벤트 저장소
```

3.2 지원 모듈

```
primitives/ # 기본 타입 및 상수
├── bytes/ # 바이트 유틸리티
├── crypto/ # 암호화 프리미티브
├── math/ # 수학 유틸리티
└── constants/ # 시스템 상수

da/ # 데이터 가용성 레이어
├── blob/ # 블롭 관리
├── kzg/ # KZG 커밋먼트
└── store/ # DA 저장소

consensus-types/ # 합의 레이어 타입
└── types/ # 블록, 상태, 검증자 타입

state-transition/ # 상태 전환 로직
```

```
└── core/ # 상태 머신 구현
```

4. 이더리움과의 주요 차이점

4.1 합의 메커니즘

- **이더리움**: LMD-GHOST + Casper FFG
- **BeaconKit**: CometBFT의 Tendermint 합의

4.2 블록 타이밍

- **이더리움**: 고정된 12초 슬롯
- **BeaconKit**: 타임아웃 기반 라운드 시스템

4.3 블록 높이 관리

- **이더리움**: 슬롯 건너뛰기 가능 (빈 슬롯)
- **BeaconKit**: 순차적 높이 (CometBFT 높이 = 비콘 슬롯)

4.4 검증자 세트

- **이더리움**: 동적 검증자 세트
- **BeaconKit**: 커스텀 검증자 세트 캡 구현

5. 시스템 특성

5.1 성능 특성

- **블록 생성**: ~1초 내 완료 필요
- **블록 검증**: 빠른 구조적 검증 우선

- **상태 전환**: 결정론적 실행

5.2 내결함성

- **Byzantine Fault Tolerance**: 최대 1/3 악의적 노드 허용
- **네트워크 분할**: 2/3+ 검증자 연결 시까지 합의 중단
- **실행 클라이언트 장애**: 재시도 로직 및 지수 백오프

5.3 확장성

- **모듈형 설계**: 컴포넌트 독립적 교체 가능
- **DI 기반**: 의존성 주입을 통한 느슨한 결합
- **인터페이스 분리**: 작고 집중된 인터페이스

6. 다음 문서 미리보기

- **02. 서비스 레지스트리 및 생명주기 관리**: 서비스 시작/중지 순서와 의존성 관리
- **03. CometBFT 통합 및 ABCI 인터페이스**: 합의 엔진과의 통합 방식
- **04. 블록 생성 프로세스**: PrepareProposal부터 블록 조립까지
- **05. 블록 처리 및 검증**: ProcessProposal과 FinalizeBlock 흐름
- **06. 상태 전환 메커니즘**: 비콘 체인 상태 머신 구현
- **07. 실행 엔진 통합**: Engine API를 통한 EL 클라이언트 연동
- **08. 저장소 아키텍처**: 다층 저장소 시스템 설계
- **09. 데이터 가용성 레이어**: 블롭 처리 및 KZG 증명
- **10. 네트워크 및 P2P**: 노드 간 통신 및 동기화

이 문서는 BeaconKit의 전체적인 아키텍처를 개괄적으로 설명했습니다. 각 컴포넌트의 상세한 구현과 동작 방식은 후속 문서에서 다루겠습니다.